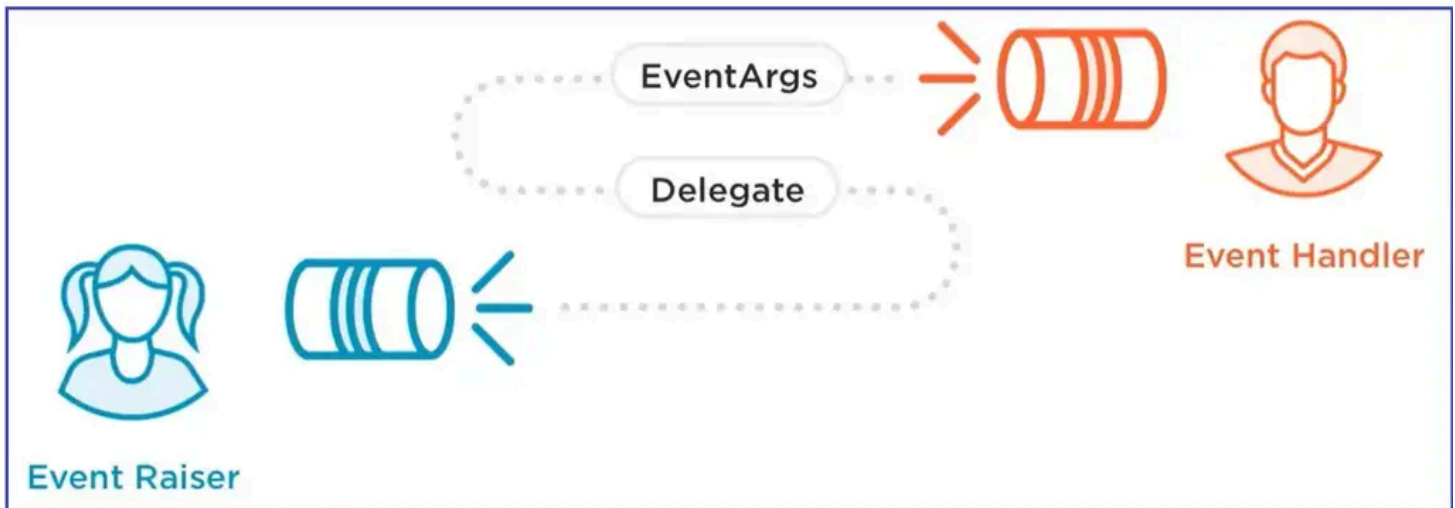


Roles of Events, Delegates, and Event Handler in C#

In this article, I am going to discuss the **Events, Delegates, and Event Handler in C#** with Examples. At the end of this article, I am sure, you will understand what are Events, Delegates, and Event Handler in C# and the role and responsibilities of Events, Delegates, and Event Handlers in the .NET Framework.

Roles of Events, Delegates, and Event Handler in C#

So, first, we will talk about the Role of Events, then we will talk about the Role of Delegates and finally, we will wrap up the role of Event Handlers. Now, a good way to understand this concept is by taking the example of **Old Tin Can** with a string analogy. If you ever did this as a kid, then you know that it was a kind of fun way to talk. This actually a really good analogy of Events, Delegates, and Event Handlers and also Event Args.



Here, on the left-hand side, we have the little girl who is talking and on the right-hand side, her dad who is listening. So, in this case, we can say that the little girl is the Event Raiser and the dad who is listening is the Event Handler. And there is a pipeline between the Event Raiser and Event Handler and we call that pipeline a Delegate. The delegate is really what makes everything possible when it comes to raising an event and handling an event. It will make the link between the Event Raiser and Event Handler and without a delegate, it is not possible to Raise an Event and also not possible to handle an Event.

So, if you think about it, if I raise an event, it means something is happening, such as the kid talking into the tin can here, then how does that actually make it over to the dad? We assume that her dad could just what the child is saying. It travels across the pipeline. That's the role of the delegate and once we progress, we will talk more about delegate.

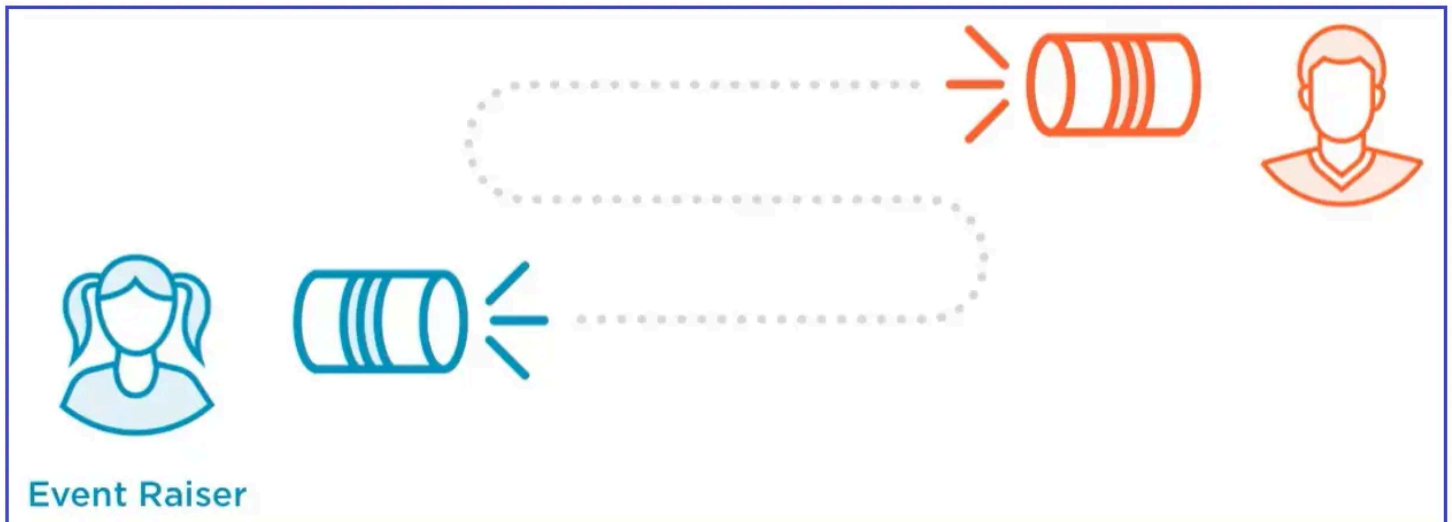
Another important piece is EventArgs because when the child is talking, we need that data or information to get over to the Event Handler i.e. whatever the child is saying should be transferred to her dad and in the .NET world, we called them as EventArgs. EventArgs are containers where we can pass one or more data. Once we progress, then we will discuss more on EventArgs.

Role of Event in C#:

Events are an important part of the .NET Framework and you can find them throughout it. For example, when you click on a button, you can go to a dropdown list, and handle the selected index changed event. Another example might be in an Ecommerce Application, when a customer places an order, an event notification

might be sent to notify other parts of the system that an order has been placed. And then they can work on that order to start the processing like shipping the order, billing the order, couriering the order, etc.

Now, let us proceed and understand the fundamental of Events in C# i.e. what are events and why they are used for? As shown earlier, an event is simply a notification such as the little girl here on the left-hand side sending a notification, in this case through speech, to the person on the right-hand side.



What is an Event?

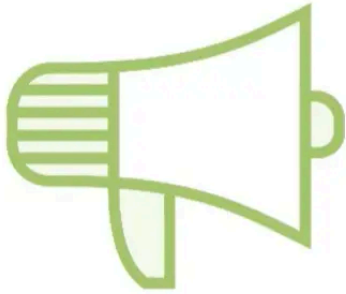
Events are notifications. In simple words, we can say that it's a message that goes out to one or more people (we can also say them as subscribers) who are listening. Obviously, it's not going to be people in .NET, it will be objects.

Events are playing a central role in .NET Framework. They are an important part of the .NET Framework and you can see them all over the place when you type anything in visual studio, if you have ever seen a lightning bulb type icon, then that's an event.

Events provide a way to trigger notifications. So, you could think of a user who talks bullhorn or loudly to announce something to a group of people that something is going to happen like the part is about to start or the present ceremony is going to start. That is also an event from the human world.

In the Programming world, an event is also the same thing, it is just that instead of having this bullhorn or instead of talking loudly, here, we are going to notify using .NET Events. And then we will also use something called Delegates, EventArgs, and EventHandlers.

What is an Event?



Events are notifications

Play a central role in the .NET framework

Provide a way to trigger notifications from end users or from objects

Example of Event:

Now, the simplest example of an event is the button. If you have worked with C# and if you have done any kind of UI work at all like Windows Form Application, and Web Form Application, then you might be known that the button element has many events like click event, mouse over, mouse out, and many more. Suppose, you drag and drop the button control from the toolbox in a Windows Form Application and place it in the UI, and when you double click on the button element, behind the scene it generates a click event handler.

In the case of a button, we have one button with one event handler that listens to the button click event. It is possible with events in C#, that multiple objects listen to a single event as long as all the objects are attached to the event, and when we sent the notifications, all the objects associated with the event will get notified.

Now, if an event is fired, we know that something happens, but typically we also have some event data that we want to know for further processing. For example, in the case of an order, let us assume the order placed event is fired, when the order placed event is fired, we would probably want to know information about the paced order like what they ordered, how much they order, and how much is the cost, where to ship it, what is the billing address, what is the shipping address, what is the payment methods, etc.

Most of the events in .NET Framework are having EventArgs or in other words, Event data gets routed from Point A (Event Raiser) to Point B (Event Handler) using the EventArgs only.

The Role of Events



Events signal the occurrence of an action/notification

Objects that raise events don't need to explicitly know the object that will handle the event

Events pass EventArgs (event data)

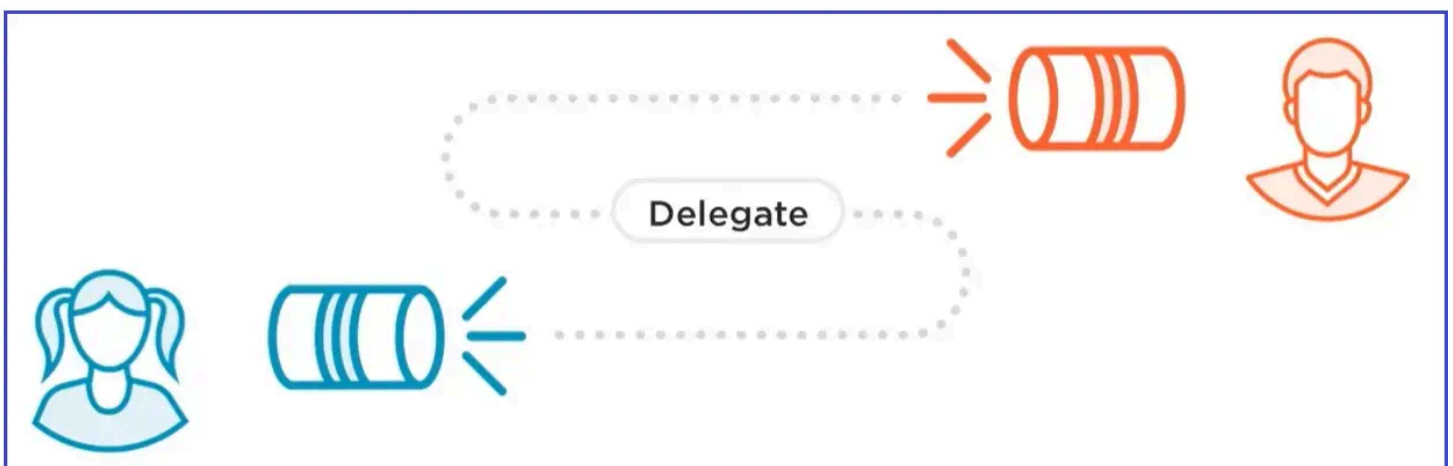
That's the role of an Event in C#.

Role of Delegates in C#:

As a programmer, we need to understand the delegate concept very well and the .NET Framework uses this delegate concept very frequently, mostly when working with autogenerated code. In Fact, without delegates, the event would not be useful at all. And the reason is that we do not have other ways to transfer the event data from point A (Event Raiser) to point B (Event Handler). So, let us proceed and try to understand what are exactly delegates and why they are used in .NET Framework.

If you go back to our TIN CAN example, Event Raiser is there who will send the notifications and of course, we have the event listener or event handler who receives the notifications. Let us imagine that when the little girl talks, she didn't talk through the TIN CAN, and let's assume that the person or the dad, who is the event listener or event handler in this case could not hear her. That means the little kid has to talk over the TIN CAN and then her speech goes over the pipeline and the person who is on the right-hand side can hear her.

So, here, in this case, the delegate is nothing but the pipeline between an Event Raiser and an Event Handler. It is one of the most important parts of the .NET Framework and most of the C# developers have lot of confusion.

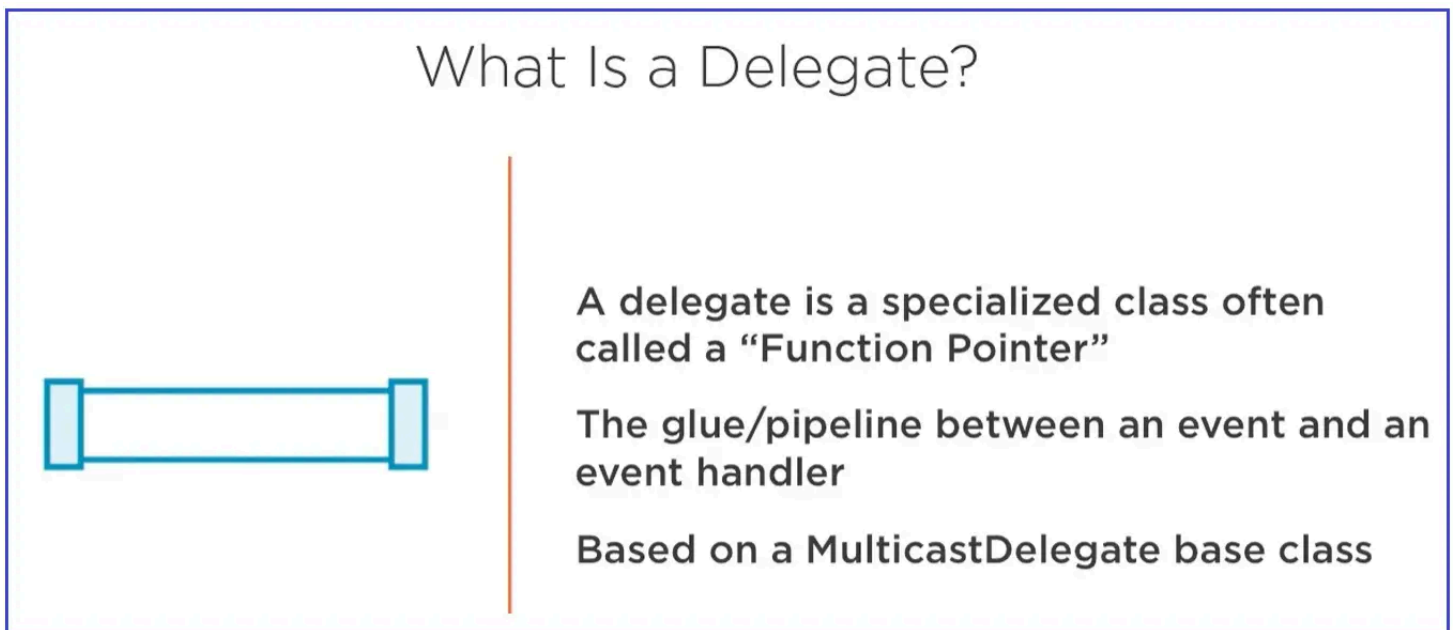


What is a Delegate Exactly in .NET Framework?

A Delegate is a specialized class, and in our coming article, you will see that there is a keyword called delegate in .NET Framework. And what really a delegate means, is a function pointer. This is the technical definition and once we start programming, you will get a better idea of what exactly a function pointer is.

But to simplify this, we can say that a delegate is a glue or pipeline between an event raiser and an event handler that allows our event and EventArgs to get over to the event handler. So, we need to transfer data from Point A to Point B somehow and this could be only possible because of the delegate.

Now, behind the scene when we use events and we created delegates. There is a special class in .NET called is MulticastDelegate and this is the class that actually tracks everyone who is listening to the event. So, when the event notification is sent, and let's say there is a piece of information, that piece of information needs to be sent to all the listeners. So, if we have 50 listeners, then we have 50 items or objects inside of what we called is Invocation list. We will understand this as progress and as we start programming in our upcoming articles.



Delegate is a Pipeline:

Delegates, as I mentioned, they are pipelines. So, we have the event, we have the pipeline and we have the event handler and we need to magically get the data from point A which is the event notification point over to the event handler which is point B.

So, what we are going to do before we raise the event, we have our EventArgs which is nothing but our event data that we want to send from Point A to Point B, and we are going to send the event data through the Pipeline. Now, the data (EventArgs) going over the pipeline and will reach the Event Handler.

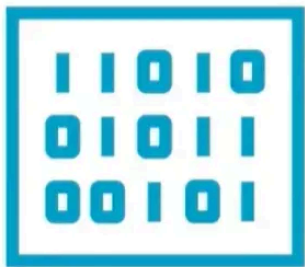
Now, it's up to the Event Handler to actually do something with the event data and process it. And we will talk about how the event handler process the data later part of this article once we talk about the role of the Event Handler.

handler wants to get some event data which we call as EventArgs. So, when the Event Raiser raises the data, the delegate routes the data to some type of call back function (Event Handler) and then the event handler processes the data and does something with the data such as updating the UI or updating the database. So, what is an Event Handler?

Event Handler is a method in C#, that is responsible for receiving and processing the data it gets from the delegate.

Normally, in C#.NET Framework the Event Handler has two parameters. The first parameter is the Sender who sent it to you and that would be an object. And the second parameter is the EventArgs object. And EventArgs is just responsible for encapsulating the data. So, it's just like a container that contains the data. So, EventArgs is just a simple way to define some properties and we can put many different types of data into those properties. Then the event handler can get the data from the EventArgs object.

What Is an Event Handler?



Event handler is responsible for receiving and processing data from a delegate

Normally receives two parameters:

- Sender
- EventArgs

EventArgs responsible for encapsulating event data

Now, let us see the role of the event handler in the bigger picture. The Event Handler is responsible for accessing the data passed by a delegate. For example, when the button is clicked, we know that the button click event is raised. Who raised the event or who sent the notification, that information will be stored inside the sender object which is the first parameter in the event handler, and then the data that we want to pass will be stored inside the EventArgs object and using the delegate the data is passed from sender to the event handler. And once the data is received by the Event Handler from the delegate, then the Event Handler can process the data. For a better understanding, please have a look at the below diagram.

An event handler is responsible for accessing data passed by a delegate:



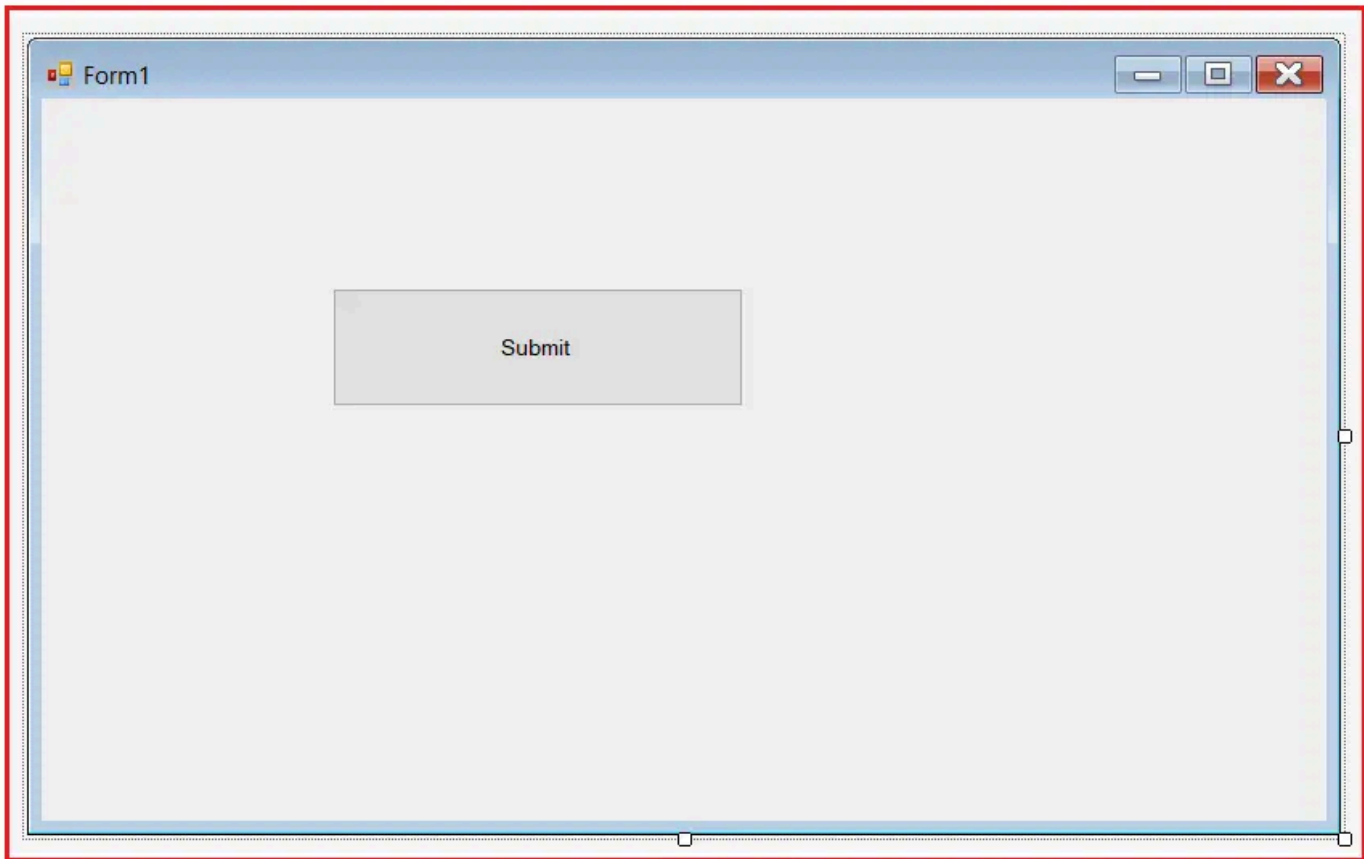
As you can see in the above example, we have one handler, in this case, we call it as btnSubmit_Click event handler. As you can see, it has two parameters i.e. object sender and EventArgs e. In this case, the EventArgs is empty and does not do anything. We just want to know that button was clicked. That's it.

But in many cases, with events, we are going to have customized EventArgs. And in our upcoming articles, I will show you how to create custom EventArgs, custom delegates, and custom events.

Example to understand Events, Delegate and Event Handler in C#:

We have discussed the concept of Events, Delegate, EventArgs, and Event Handler in C#. Now, let us see a quick example to wrap up all those things. This is just an introduction, and you might be having many confusions and doubts and that is acceptable. So, whatever we are discussing in this article, we are going to discuss those things in detail in our upcoming articles and I hope once you go through those articles, you will get better clarity on the Events and Delegates concept.

Let us create a Windows Form Application with one button. Just drag the button control from the toolbox and drop it on the UI. The UI should look as shown in the below image. Here, you can see that I have changed the button text to Submit using the properties of the button control.



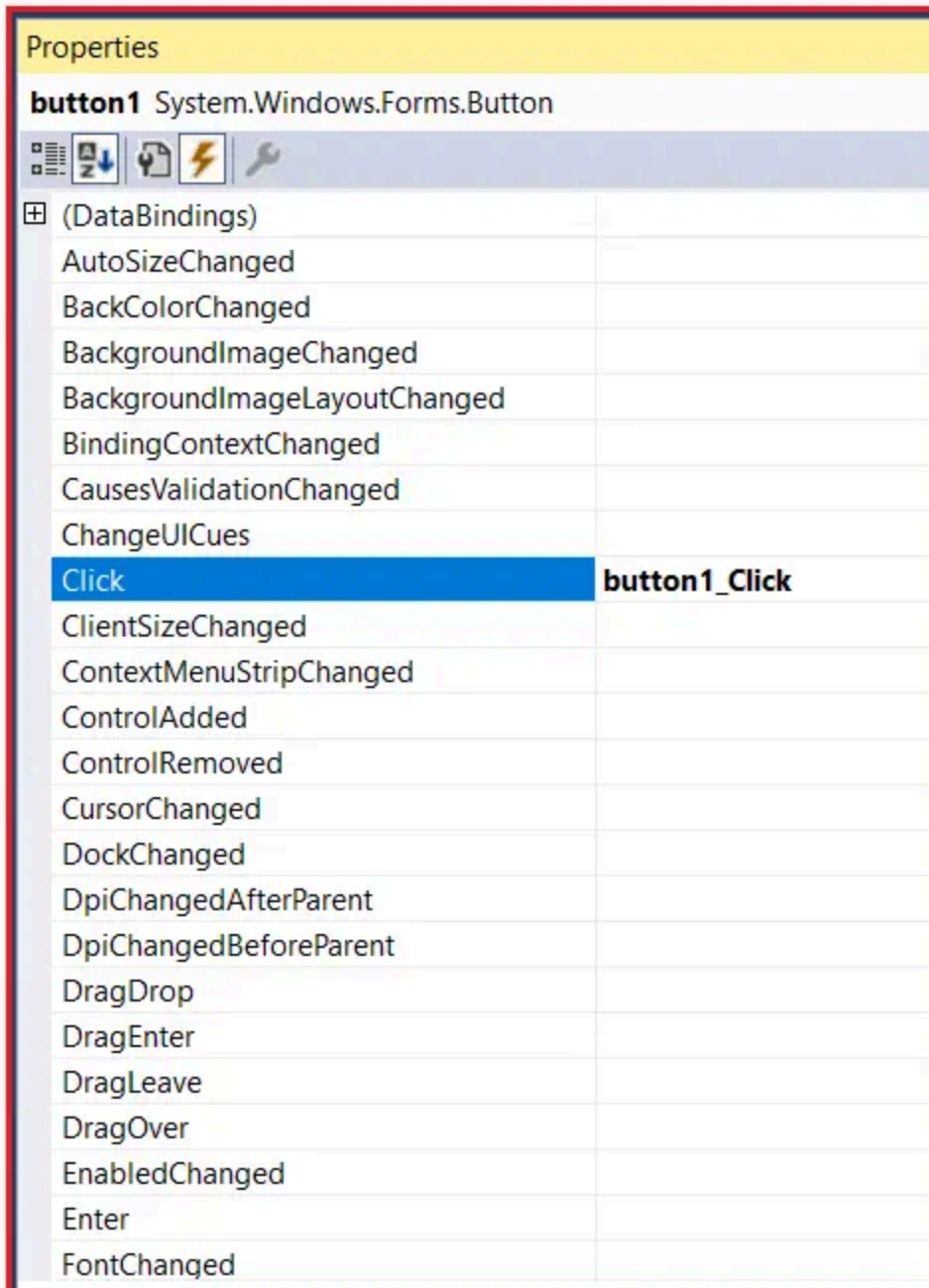
Once you double click on the button, the following Event Handler is going to be generated for the button click event and you can see that it is a method taking two parameters i.e. object sender and EventArgs e. The sender object contains the information about who raised the event, in this case, it is the submitted button. And EventArgs property contains the data.

```
using System.Windows.Forms;
namespace WindowsFormsApp2
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

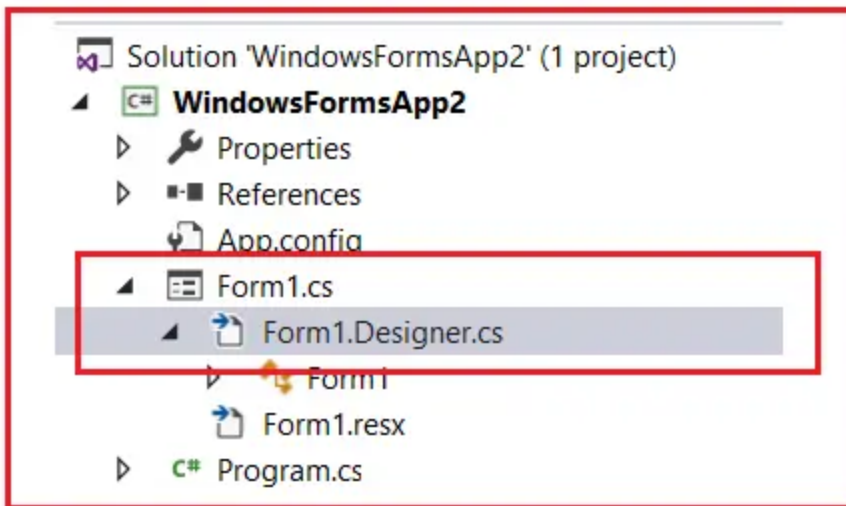
        private void button1_Click(object sender, EventArgs e)
        {
        }
    }
}
```

↑ Event Handler

Now, if you select the button control and go to the properties window then you can see all the events that are associated with the button element shown in the below image.



And out of all the events, in our example, we have only added the handler for the click event. So, we have seen the events of the button element and we have seen the event handler i.e. the click event handler. But where is the delegate? If you want to see the delegate, then expand Form1.cs class file and there you will see Form1.Designer.cs as shown in the below image.



Now, open Form1.Designer.cs class file and this file is generated when we double click on the button element on the UI. Now, expand the Windows Form Designer generated code section and you will see the following.

```
#region Windows Form Designer generated code
```

```
/// <summary>
/// Required method for Designer support - do not modify
/// the contents of this method with the code editor.
/// </summary>
private void InitializeComponent()
{
    this.button1 = new System.Windows.Forms.Button();
    this.SuspendLayout();
    //
    // button1
    //
    this.button1.Location = new System.Drawing.Point(181, 118);
    this.button1.Name = "button1";
    this.button1.Size = new System.Drawing.Size(256, 74);
    this.button1.TabIndex = 0;
    this.button1.Text = "Submit";
    this.button1.UseVisualStyleBackColor = true;
    this.button1.Click += new System.EventHandler(this.button1_Click);
```

Button Click Event

Delegate

Callback Function

As you can see in the above image, the button click event is wrapped by the delegate and that delegate is pointing to the button1_click call back function. So, what all these are and how they are working, we are going to discuss them in detail in our coming session. This is just an introduction to Event, Delegates, and Event Handler. So, the point that you need to remember is that if you are using an event, then behind the scene delegate is there.

Summary of Events, Delegates, and Event Handlers in C#:

1. The .NET Framework relies heavily on Events and Delegates.
2. Events provide notification and send that using EventArgs.
3. The delegate acts as the Glue or Pipeline between the Event and Event handlers.
4. Event Handlers receive and process EventArgs data.

That's it for today. In our next article, we are going to discuss [Creating Delegates, Events, and EventArgs in C#](#) with Example. Here, in this article, I try to explain what are **Delegates, Events, and Event Handler** and their role and responsibilities in C#. I hope you enjoy this Event, Delegates, and Event Handlers in C# article.